Tower Defense Game

Software Requirements Specifications (SRS)

Version: 1.0

Authors: Sejin Kim

Alejandro Pantoja-Zurita

Alejandra Torres

Connor Beachy

Raheem McGowan

School: Delaware Technical Community College

Class: CIS 210: Computer Science III

Instructor: Tianzhong Ding

Team: Group 1

Table of Contents

1. Preface

Version	Date	Version Description
1.0	10/13/25	Initial Software Requirements Specifications Worksheet Created and submitted.

2. Introduction

The Software Requirements Specification (SRS) is designed to document and describe the specification of the *Tower Defense Game* to be created. It will provide a clear and descriptive “statement of user requirements” that can be used as a reference in the development of the proposed game. This document is divided into several sections to logically organize the software requirements into easily accessible parts.

This SRS aims to describe the functionality, external Interfaces, attributes, and design constraints imposed on the implementation of the software system described throughout the rest of the document.

1.1 Purpose

The purpose of this Software Requirements Specification (SRS) is to define the functions and specifications for the proposed software. This document is intended to provide a detailed description of the system's capabilities, constraints, and interactions with external entities. It serves as a reference for the project stakeholders including the development team, testers, and project managers to ensure a shared understanding of the system requirements.

1.2 Scope

The software system shall define a 2D Tower Defense Game, in which a single player is tasked with defending their base against waves of enemies, and avoiding their health running out. The player shall use various strategies to mitigate the enemies from reaching the end of the path, towards their base. The primary objective of a level shall be to survive the waves of enemies successfully while managing health and in-game

currency. The final goal shall be to clear all levels successfully, thus completing the game. As the game progresses, the player shall receive in-game rewards for completing levels, which can be used to upgrade their defenses. The game shall include systems such as 2D graphics, enemy pathfinding, player health, scoring, and adaptive music. Players shall also be able to adjust difficulty settings through the settings menu. The game will not include 3D graphics, online features, or multiple players.

1.3 Document Overview

The subsequent section(s) of this document describe the Tower Defense Game in detail as it is related to User Requirements, System Requirements & Specifications. The remaining sections include a Glossary, User Requirements, System Architecture, System Models, and Index. The Glossary includes definitions for terms and acronyms used throughout the documentation. The User Requirements section will specify the functional requirements and non-functional requirements that address the player's perspective that describes expected gameplay elements and interactions with the game. The System Architecture section will present all of the components of the software overall and their relationship hierarchies. The System Requirements section will detail the technical specifications and include constraints and performance parameters of the system, where it may be complex in nature due to various technologies such as requirements from the game engine. The System Models section will include diagrams and models, representing various interactions of how the various parts of the system (software and users) interact. Finally, the Index provides an outline for navigating important topics and terms, based on a published index, to observe other important sections.

3. Glossary

Term/ Acronym	Description
API	Application Programming Interface – a set of routines, protocols, and tools for building software
CRUD	Create, Read, Update, Delete – the basic operations of persistent storage.
GUI	Graphical User Interface – the visual part of the application that the user interacts with
User	A person who interacts with the system. May be classified as "Admin", "Editor", or “Player”
SRS	Software Requirements Specification – this document.
Level	A specific area where the game takes place.Can also be known as a “Map”
Tower	An entity the player can place in a specific location and will fulfill a task autonomously.
Wave	A group of enemies that are released together with a break between waves. Can also be known as a “Round”.
Health	Tracks the remaining health of the player base and enemies. Player health decreases when enemies reach the base; enemy health decreases when towers attack them.
Currency	In-game resource that players earn by defeating enemies and can spend to place or upgrade towers

Upgrades	Mechanic for improving tower stats, abilities, or other player advantages using game currency.
NPC	Non-Player Character
HUD	(Heads-Up display) On-screen display of essential game information, such as player health, currency, score, and wave number
Saving	A mechanic that allows the game to keep progress, including levels completed, currency, and upgrades, across sessions
Leaderboard	A system that records and displays high scores or player rankings
Adaptive Music	Background music that changes dynamically based on events happening

4. User Requirements

4.1 Functional Requirements

Title: Towers

Description: This feature lets players place, upgrade, and manage towers that automatically attack approaching enemies. There are different types of towers that vary in power, range, and abilities, which allows players to strategize during gameplay

Rationale: Towers are literally the foundation of tower defense games. They enable players to strategically defend their base, experiment with layouts, and adapt to different enemy types.

Dependency: Requires enemy pathfinding, attack mechanics, currency system, and grid placement logic

Priority: High - Core gameplay component needed for the base game and testing

---

Title: Enemies

Description: -Enemies are entities that will start at the beginning of a path and move to the end of the path. They will do damage to the player when they get to the end of the path.

Rationale: Provides a goal for the player.

Dependency: A path system for the enemies to follow, a way of dealing damage to the enemies, and a health system for the player.

Priority: High - Fundamental system within the game.

Title: Upgrade System

Description: This system allows the player to make decisions about how to spend currency obtained from defeating enemies in order to improve the performance of their defenses throughout a level.

Rationale: The player must decide what to upgrade within a level. It reinforces awareness and focus on the game.

Dependency: This upgrade system requires enemies and a form of eliminating these enemies for the system to be implemented. A system must be in place to calculate the amount of damage dealt to enemies initially so that upgrades can further improve damage dealt.

Input: A pop-up will appear at the end of each round within a level with a set of upgrades available to the player to choose from. The player will be prompted to select from the options or skip the upgrade

Output: The system will determine if an upgrade is obtainable once selected, based on the amount of currency the player currently has. If it is not sufficient, the system will notify the user that they are unable to buy the upgrade.

Priority: Medium - Not required to create an initial demo of the software.

Title: Difficulty Settings

Description: A settings option that will determine how many enemies will spawn, and other properties, in order to change how difficult it is for the player to succeed.

Rationale: Will make it so the difficulty of the game will be customizable, so that players can play at a difficulty that suits them.

Dependency: enemies, upgrades.

Priority: High - Fundamental system within the game.

Title: Save System

Description: This system allows the player to have progress, such as level completion, score, and currency upgrades to be saved and loaded. Saving can either be automatically done in increments or manually saved by the player.

Rationale: A save system is a good addition as it allows the player to continue his progress from his last time running the game and encourages longer engagement.

Dependency: Requires access to player score, level, player stats such as currency and upgrades. Depends on a database to store and retrieve data.

Priority: Medium - Not necessarily needed for an early demo but required for full release

---

Title: Main Menu & Opening Sequence

Description: A system that displays when the game starts. Options to start/load a play file, open settings, view credits and exit. The opening sequence may have a short animation or title screen.

Rationale: Provides a structured entry to the game and initial impression. Essential for navigation between game modes.

Dependency: Requires functioning UI, save system for Load Game and settings menu.

Priority: High - Needed for any playable demo.

Title: Settings Menu

Description: This system allows players to customize game options to their liking. It will include an audio volume and difficulty level. It allows users to play the game more to their personal preference.

Rationale: A settings menu improves user experience by giving the player more control over their experience with the game

Dependency: Requires a simple UI system with sliders and toggles.

Priority: Medium - Not required for a demo, but when the difficulty level is made, this is a requirement.

Title: Health System

Description: This system will keep track of and update the player’s and enemies' remaining health during gameplay. Player health decreases when enemies breach main base. Reaching zero health triggers a game-over screen. Enemy health decreases when towers do damage to them. When enemy health reaches zero, they are removed.

Rationale: The health system encourages players to plan their moves, which raises tension. It impacts the progression and difficulty of the game.

Dependency: Requires functioning enemy attack logic, tower attack logic, collision detection, and UI to display health.

Priority: High - Core gameplay component needed for the base game and testing.

4.2 Non-Functional Requirements

Title: Game Currency

Description: This system will track the currency available to the player at any given time with a level. The currency can be spent on upgrades to help the player defend their base from incoming enemies. Players will be awarded more currency to spend upon defeating enemies.

Rationale: The player must engage with this system in order to survive enemy attacks. This serves as a fundamental pillar to the game loop within a level. It reinforces awareness and focus on the game

Dependency: Game currency requires the implementation of an upgrade system for the user to spend the currency on. It also requires enemies and a means of eliminating these enemies for the system to be implemented.

Input: The system will check if an enemy has been defeated as a flag to update player currency.

Output: Upon defeating an enemy, the system will update current player currency to reflect currency earned from defeating an enemy.

Priority: High - Fundamental system within the game.

Title: Adaptive Music

Description: A system to dynamically change the background music depending on what state the game is in.

Rationale: Will help with establishing atmosphere and will help the player with knowing what is happening in the game.

Dependency: An ability to detect the changes that can affect the music.

Priority: Medium - Not required to create an initial demo of the software.

Title: Non-Playable Characters (NPCs)

Description: The system shall incorporate NPCs to follow dynamic paths from the spawn point, to the target. Dialogue systems may be incorporated depending on the role of the NPC.

Rationale: Ensures game enemies or other characters have a path to reach the base of the player for the game to proceed.

Dependency: Pathfinding system, spawn points, tower placement system.

Input: The system will require a player's base location, enemy spawn points, and a coherent map layout.

Output: The system shall produce enemy movement along a path. As for other non-antagonistic NPCs, dialogue could be produced.

Priority: High - Fundamental game mechanic.

Title: 2D Visuals

Description: The game shall use 2D graphics for all visual features, including characters, environments, and user interface components. There shall be visual consistency and clarity to ensure readability and aesthetic appeal throughout the gameplay.

Rationale: 2D visuals align with the design and purpose of the project, which will provide a balance between performance & visual appeal. They will also simplify asset

creation or animations, making it easier to keep a consistent art style, ensuring smooth performance.

Dependency: Artistic assets such as effects, sprites, and backgrounds. A rendering engine to display the 2D elements. An animation system for characters, objects, and buttons.

Priority: High - Fundamental system within the game.

Title: “Game Over” Screen

Description: A “Game Over” Screen shall appear when the player loses all health or fails to complete an objective. It shall display a “Game Over” message, the player’s score, and options to restart the level, return to the main menu, or exit the game.

Rationale: A “Game Over” screen provides closure to the playing session and gives the player information about their performance. In addition to that, the screen is intuitive for players to navigate without confusion, enhancing the user experience.

Dependency: A scoring and health system is required to close the gaming session and display performance. Main menu system. A system to detect when the game ends.

Priority: Medium - Not required to create an initial demo of the software.

Title: End Credits

Description: An end credits screen shall appear after the player successfully finishes the final level or stage of the game. It shall display the names of the team members, contributors, or any relevant acknowledgments. The credits may also include scrolling text, visuals, or background music.

Rationale: An end credits screen recognizes the efforts of all team members and will provide a sense of accomplishment for the player.

Dependency: 2D visual system for scrolling text effects, audio system for background music, and a level progression system that allows the game to conclude.

Priority: Low - Not required to create an initial demo of the software, optional feature.

Title: Leaderboard System

Description: The Leaderboard System links the game up to an external or local data source in order for players rankings to upload, retrieve and display. It communicates via UI panels that have player names and score display at all times, updating dynamically either through stored results or the online results.

Rationale: Enables competitive play and encourages player engagement by ranking each other's scores against each other.

Dependency: Requires UI to display the leaderboard and a data storage mechanism.

Priority: Medium – Not required for initial demo but valuable for final version for replayability.

4.3 External Interface Requirements

Title: Player Input (Keyboard & Mouse)

Description: The game receives input from the player from keyboard and mouse. Mouse input is used for tower placement, selection, and menu navigation. Keyboard input is used for shortcuts like pausing

Rationale: Player input is necessary for interacting with the game and making gameplay decisions.

Dependency: Requires UI and game logic systems to detect and process input.

Priority: High – Core interface for player interaction

Title: UI (User Interface)

Description: Gives all visual and interactive elements that allow the user to navigate/interact with the game. Includes the Main menu, settings menu, HUD, and game over screen. By interacting with the UI, users can start or load a game, adjust the settings, and view progress in game.

Rationale: UI allows users to navigate and interact with all parts of the game. It gives the user a better experience by having clear and intuitive control over gameplay and system functionality.

Dependency: Requires a functional UI framework in Godot, an event-handling system, and access to the state of the game. Also depends on save/load system to keep settings and progress

Priority: High – Needed so user can interact with the game

---

5. System Architecture

6. System Requirements

7. System Models

8. Index